

ISWE406P – Agile Development Process & DevOps

Assessment – VII

Slot: L 19+20

For the program - Student Management and Academic Performance System implement any three Jenkins CI/CD projects using any programming language or development tool of your choice. (The Tutorial is attached below).

Student instructions

1. Select any programming language, framework, or development environment suitable for your implementation.
2. Create a separate GitHub repository for each project or use one repository with clearly separated project folders.
3. Implement the required Jenkins pipeline concepts.
4. Execute the pipeline in Jenkins and demonstrate the output.
5. Submit the source code, Jenkins file, screenshots, and a brief explanation of the implementation.

Jenkins CI/CD Lab — 5 More Pipeline Projects (Windows Agent)

Course: ISWE406L — Agile Development Process and DevOps All five projects share **Stage 1: Checkout from GitHub**. Each project teaches a concept not covered in the first set of five (build & test, parametrized tests, fixtures, manual approval, built-in environment variables, linting, post-build success/failure).

Prerequisites (set up once, before class)

1. **Jenkins** installed and running on a Windows machine, with the **Git plugin** installed.
 2. **Python** installed on the Jenkins agent, added to PATH (verify with `python --version` in cmd).
 3. Each student has a **GitHub account** and a **public repository** for their project.
 4. In Jenkins: **New Item** → **Pipeline**, and either paste the Jenkinsfile directly into the **Pipeline script** box, or use **Pipeline script from SCM** pointing at the student's GitHub repo (with **Jenkinsfile** committed at the repo root).
-

PROJECT 1: Parameterized Build Pipeline

Concept taught: pipelines don't have to run the same way every time — Jenkins can ask for input **before** the build even starts, using the **parameters** block. This is different from the manual approval gate from the earlier set (which pauses mid-run); here, the choice is made up front, before Stage 1 runs at all.

Files to create in the GitHub repo

Jenkinsfile

```
pipeline {
    agent any

    parameters {
        choice(name: 'ENVIRONMENT', choices: ['dev', 'staging', 'prod'], description: 'Select the deployment environment')
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Show Parameter') {
            steps {
                echo "Selected environment: ${params.ENVIRONMENT}"
            }
        }

        stage('Build for Environment') {
            steps {
                echo "Building the application for the ${params.ENVIRONMENT} environment..."
            }
        }
    }
}
```

No `app.py` is needed for this one — the whole point is the parameter, not the code.

Execution procedure

1. Create a new GitHub repo, add just the `Jenkinsfile`, and push.
 2. In Jenkins, create a new Pipeline job pointing to this repo.
 3. Click **Build Now** the **first time only** — Jenkins doesn't know about the `parameters` block until it has read the `Jenkinsfile` once, so the very first run happens without asking for input.
 4. From the second run onward, the job's sidebar button changes to **Build with Parameters**. Click it — you'll now see a dropdown asking you to pick `dev`, `staging`, or `prod` before the build starts.
 5. Pick `staging`, run it, then open **Console Output** — confirm both echoed lines show `staging`.
 6. Run it again picking `prod` — show that the exact same `Jenkinsfile` now behaves differently, purely because of the value chosen at trigger time.
-

PROJECT 2: Archive Build Artifacts Pipeline

Concept taught: Jenkins can save files a build produces so they can be downloaded later from the Jenkins UI, using `archiveArtifacts`. This is the first project where the *output* of a pipeline run — not just pass/fail status — is something students can click and open.

Files to create in the GitHub repo

`app.py`

```
with open("report.txt", "w") as f:
    f.write("Application Report\n")
    f.write("Total Users: 120\n")
    f.write("Active Sessions: 45\n")

print("Report generated.")
```

`Jenkinsfile`

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Generate Report') {
            steps {
                bat 'python app.py'
            }
        }

        stage('Archive Report') {
            steps {
                archiveArtifacts artifacts: 'report.txt', fingerprint: true
            }
        }
    }
}
```

```
}  
}
```

Execution procedure

1. Create a new GitHub repo, add `app.py` and `Jenkinsfile`, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**.
4. Open the completed build's page (not the console output — the main build page). Look for a section called **Build Artifacts** or **Artifacts** listing `report.txt`.
5. Click on `report.txt` — it opens directly in the browser, showing the contents `app.py` generated.
6. Change the numbers in `app.py` (e.g. `Total Users: 200`), push, rebuild — show that each build keeps its **own separate copy** of the artifact, so old builds' reports are still available even after a newer build runs (click back to an older build number and confirm its artifact still shows the old numbers).

PROJECT 3: Parallel Stages Pipeline

Concept taught: pipeline stages don't have to run one after another — independent tasks can run **at the same time** using the `parallel` block. This introduces the idea that pipelines can be made faster by not waiting on tasks that don't depend on each other.

Files to create in the GitHub repo

`frontend_check.py`

```
import time  
  
print("Running frontend checks...")  
time.sleep(3)  
print("Frontend checks passed.")
```

`backend_check.py`

```
import time  
  
print("Running backend checks...")  
time.sleep(3)  
print("Backend checks passed.")
```

`Jenkinsfile`

```
pipeline {  
    agent any  
  
    stages {  
        stage('Checkout') {  
            steps {  
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'  
            }  
        }  
  
        stage('Parallel Checks') {  
            parallel {  
                stage('Frontend Check') {  
                    steps {
```

```

        bat 'python frontend_check.py'
    }
}
stage('Backend Check') {
    steps {
        bat 'python backend_check.py'
    }
}
}

stage('Summary') {
    steps {
        echo 'Both frontend and backend checks are complete.'
    }
}
}
}

```

Execution procedure

1. Create a new GitHub repo, add `frontend_check.py`, `backend_check.py`, and `Jenkinsfile`, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**.
4. If your Jenkins has the pipeline visualization view (stage view / Blue Ocean), point out that “Frontend Check” and “Backend Check” appear **side by side**, not one after the other, and both start at roughly the same time.
5. Open **Console Output** — note that lines from both scripts are interleaved rather than one script’s output fully finishing before the other starts.
6. Do the math with students: each script sleeps for 3 seconds. If they ran one after another, the stage would take roughly 6 seconds. Because they run in parallel, the whole stage finishes in roughly 3 seconds — the time of the *slower* one, not the *sum* of both. This is the practical payoff of running independent work concurrently.

PROJECT 4: Conditional Stage Execution Pipeline

Concept taught: a stage can be told to **only run under certain conditions**, using the `when` directive. This introduces conditional logic inside a pipeline — some stages are not always relevant to every run.

Files to create in the GitHub repo

`app.py`

```

def greet(name):
    return "Hello, " + name

```

`Jenkinsfile`

```

pipeline {
    agent any

    parameters {
        booleanParam(name: 'RUN_EXTRA_CHECK', defaultValue: true, description: 'Run the extra check stage')
    }
}

```

```

}

stages {
    stage('Checkout') {
        steps {
            git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
        }
    }

    stage('Build') {
        steps {
            bat 'python -m py_compile app.py'
            echo 'Build successful: app.py compiled with no syntax errors'
        }
    }

    stage('Extra Check') {
        when {
            expression { params.RUN_EXTRA_CHECK == true }
        }
        steps {
            echo 'Running extra check: verifying greet() output format...'
            bat 'python -c "from app import greet; print(greet(\'Student\'))"'
        }
    }
}
}

```

Execution procedure

1. Create a new GitHub repo, add `app.py` and `Jenkinsfile`, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now** the first time (registers the `RUN_EXTRA_CHECK` parameter, same reason as Project 1).
4. Click **Build with Parameters**, leave the checkbox for `RUN_EXTRA_CHECK` **checked** (true), and build — confirm the “Extra Check” stage runs and prints its output.
5. Click **Build with Parameters** again, this time **uncheck** `RUN_EXTRA_CHECK`, and build — show that the “Extra Check” stage now appears **greyed out** / **skipped** in the pipeline view instead of running.
6. Discuss with students: the **when** block was evaluated before deciding whether to even attempt the stage — this is different from a stage running and then failing; a skipped stage never runs its steps at all.

PROJECT 5: Custom Environment Variables Pipeline

Concept taught: the earlier set covered Jenkins’ **built-in** environment variables (`BUILD_NUMBER`, `JOB_NAME`, `WORKSPACE`). This project introduces **custom** ones — variables the student defines themselves in an **environment** block, which then become available to every stage in the pipeline.

Files to create in the GitHub repo

`app.py`

```

def add(a, b):
    return a + b

```

Jenkinsfile

```
pipeline {
    agent any

    environment {
        APP_NAME = 'GradeBookApp'
        APP_VERSION = '1.0.0'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Show App Info') {
            steps {
                echo "Building ${env.APP_NAME}, version ${env.APP_VERSION}"
            }
        }

        stage('Build') {
            steps {
                bat 'python -m py_compile app.py'
                echo "${env.APP_NAME} version ${env.APP_VERSION} compiled successfully."
            }
        }
    }
}
```

Execution procedure

1. Create a new GitHub repo, add `app.py` and `Jenkinsfile`, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**.
4. Open **Console Output** — confirm `APP_NAME` and `APP_VERSION` appear correctly in both the “Show App Info” and “Build” stages, even though they were only defined once, at the top of the file.
5. Change `APP_VERSION` to `'2.0.0'` in the `Jenkinsfile`, push, and rebuild — show that **both** stages automatically pick up the new value, since they both reference the same `environment` block rather than having the version typed out separately in each stage.
6. Compare with the earlier built-in environment variables project: ask students what’s different — here, *they* chose the variable names and values; with `BUILD_NUMBER` or `WORKSPACE`, Jenkins decided both automatically.

Summary table

	Concept	Stage 2	Stage 3
Project 1	Parameterized builds (input chosen before the run starts)	Show Parameter	Build for Environment

	Concept	Stage 2	Stage 3
Project 2	Archiving build artifacts for later download	Generate Report	Archive Report
Project 3	Parallel stage execution	Parallel Checks (Frontend + Backend)	Summary
Project 4	Conditional stage execution (when)	Build	Extra Check (conditional)
Project 5	Custom (user-defined) environment variables	Show App Info	Build

Contrast with the first set of five: those covered automated testing (basic, parametrized, and with fixtures), Jenkins' built-in environment variables, code linting, a manual mid-pipeline approval gate, and post-build success/failure hooks. This set instead covers **how a pipeline can be shaped by choices** — parameters chosen up front, conditions checked mid-run, tasks split to run concurrently, custom variables threaded through stages, and outputs preserved as artifacts.